CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
Approved by | Affiliated to | UGC Autonomous | 14 Programs Accredited by NBA | Grade A++ in NAAC | All India Ranking 151-200 Band nirf | Certifications Quality ISO 21001 : 2018 Green ISO 14001 : 2015 Energy ISO 50001 : 2018
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

ENGINEERING CASE STUDY REPORT

Student Performance Prediction Using Fuzzy Logic

Design and Implementation of a Mamdani Fuzzy Inference System

STUDENT DETAILS	
Name	Prajeeth Reddy
Roll no.	160124748127
Faculty	Dr. Y.C.A. Padmanabha Reddy (Associate Professor)

Department of Engineering
Chaitanya Bharathi Institute of Technology
Hyderabad, Telangana

Abstract

This case study presents the design and implementation of a fuzzy-logic-based student performance prediction system. The model evaluates three academic indicators—attendance, marks and daily study hours—and maps them to one of four performance categories: poor, average, good or excellent. A Mamdani Fuzzy Inference System (FIS) is implemented in Python using triangular membership functions, an interpretable IF–THEN rule base, minimum-based rule evaluation, maximum-based aggregation and centroid defuzzification. The accompanying illustrative case study contains four students. Running the supplied program produces scores of 89.6, 43.0, 13.6 and 67.1 for students A–D respectively, matching the categories shown in the presentation. These values demonstrate the implementation; they are not results from a real student dataset or a controlled experiment.

1. Introduction

Student performance is influenced by multiple factors rather than examination marks alone. Attendance, academic marks and study habits can jointly indicate a student's progress. These factors may also be qualitative or uncertain: for example, attendance can be described as low, medium or high, while marks can be considered poor, average or good. A rigid threshold-based system may not represent such gradual transitions effectively.

Fuzzy logic provides a way to model these linguistic descriptions. Instead of assigning an input to only one sharply bounded class, a fuzzy system allows it to belong to neighbouring classes with different degrees of membership. The resulting rules are readable and can be inspected to understand how an output was produced.

2. Problem Statement

The engineering problem addressed is to combine attendance, marks and study hours into a single interpretable estimate of academic performance. The system must handle overlapping input categories and provide both a numerical score on a 0–100 scale and a linguistic performance label.

3. Objectives

- Represent attendance, marks and study hours using fuzzy linguistic variables.
- Create an IF–THEN rule base that maps combinations of input conditions to performance categories.
- Implement Mamdani inference and centroid defuzzification in Python.
- Evaluate the model using the four illustrative students in the presentation.
- Make the decision process explainable by exposing membership degrees and fired rules.

4. System Overview and Architecture

The implemented processing pipeline is: crisp input values → fuzzification → rule evaluation → implication → aggregation → defuzzification → predicted score and category. The three inputs are attendance (0–100%), marks (0–100) and study hours (0–8 hours/day). The output is performance (0–100), represented by four fuzzy sets.

Stage	Function in the system
Crisp inputs	Accept attendance percentage, marks and daily study hours.

Fuzzification	Calculate each input's degree of membership in its linguistic terms.
Rule evaluation	Evaluate the IF conditions; AND is implemented using the minimum operator.
Implication	Clip the consequent membership function at the rule firing strength.
Aggregation	Combine the rule outputs using the maximum operator.
Defuzzification	Convert the aggregated fuzzy output into a crisp score using the centroid.
Decision	Assign the output label with the highest membership at the crisp score.

5. Fuzzy Variables and Membership Functions

The code defines triangular membership functions. A triangular set rises linearly from zero at its left endpoint to one at its peak, then falls linearly to zero at its right endpoint. Degenerate triangles are used as shoulder sets for open-ended low and high categories. Overlap allows a value to partially belong to adjacent categories.

Variable	Universe	Terms and breakpoints
Attendance	0–100 %	Low (0, 0, 50); Medium (25, 50, 75); High (50, 100, 100)
Marks	0–100	Poor (0, 0, 50); Average (25, 50, 75); Good (50, 100, 100)
Study hours	0–8 h/day	Low (0, 0, 4); Medium (2, 4, 6); High (4, 8, 8)
Performance	0–100	Poor (0, 0, 35); Average (20, 40, 60); Good (45, 65, 85); Excellent (70, 100, 100)

Important implementation note: the presentation specifies the linguistic categories but does not provide numerical membership-function breakpoints. The code therefore uses reasonable default breakpoints that can be calibrated later. These are implementation assumptions, not measured or validated thresholds.

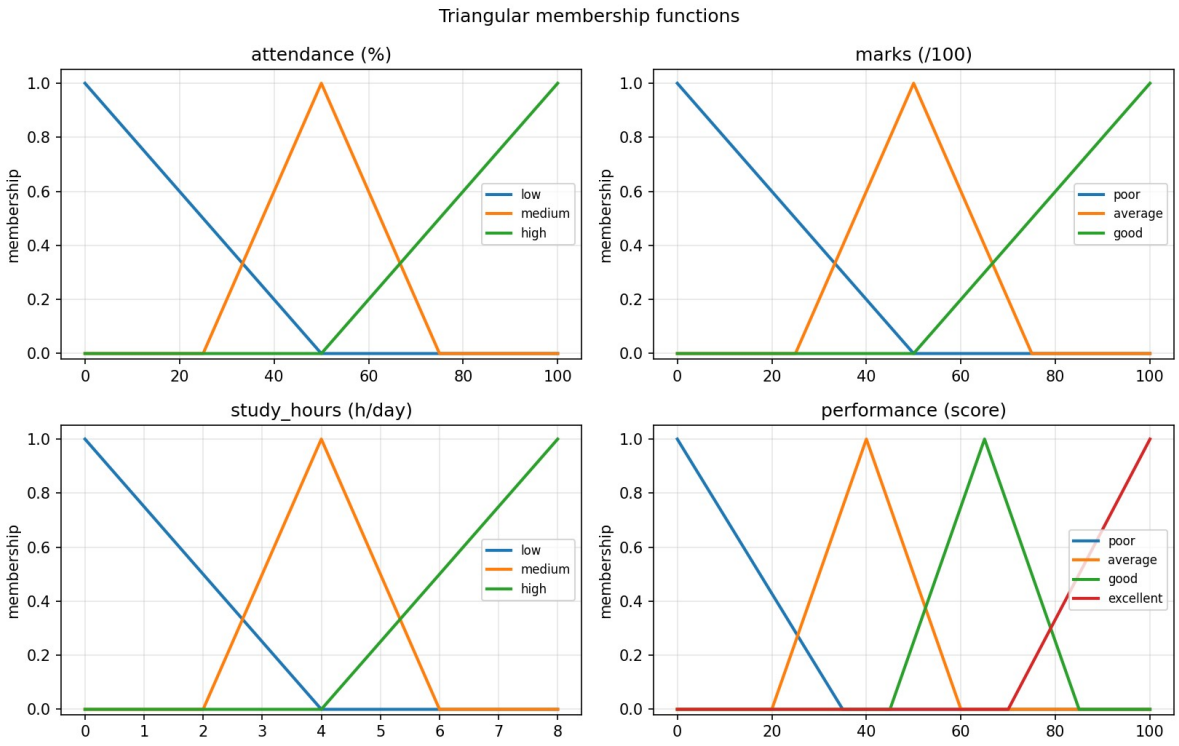


Figure 1. Triangular membership functions used by the implementation.

6. Rule Base and Inference Method

The presentation specifies four explicit rules:

- IF attendance is high AND marks are good THEN performance is excellent.
- IF attendance is medium AND marks are average THEN performance is average.
- IF attendance is low AND marks are poor THEN performance is poor.
- IF study hours are high AND marks are good THEN performance is excellent.

To cover combinations beyond these four rules, the code generates additional rules. It assigns relative importance weights of 0.50 to marks, 0.30 to attendance and 0.20 to study hours. Linguistic terms are converted to ordered levels (low/poor = 0, medium/average = 1, high/good = 2). A weighted composite value is then mapped to an output category using cut-offs at 0.30, 0.60 and 0.85. The explicit slide rules take precedence over generated rules.

Rule-base detail	Implemented value
Explicit rules from presentation	4
Generated rules	16
Total rules in Python implementation	20
Possible full input combinations	3 × 3 × 3 = 27
Inference type	Mamdani
AND / implication	Minimum
Aggregation	Maximum

Defuzzification	Centroid

The accompanying research article cited in the Python file discusses a different configuration with four inputs and 81 rules (3^4). This implementation uses the three inputs defined in the presentation and has 20 rules (four explicit plus 16 generated). The two rule counts should not be treated as identical.

7. Implementation Details

The program is implemented in Python. NumPy is the required numerical dependency; Matplotlib is optional and is used to generate membership-function and control-surface plots. The code is organised into fuzzy-set primitives, linguistic variables, rule representation, a Mamdani inference engine, model construction, the illustrative case study, an explanation routine, plotting utilities and a command-line interface.

Component	Purpose
trimf()	Computes triangular membership values.
LinguisticVariable	Stores a variable's range and fuzzy terms; supports fuzzification.
Rule	Stores antecedents, consequent, weight and rule source.
MamdaniFIS.infer()	Performs fuzzification, rule firing, implication, aggregation and defuzzification.
build_variables()	Defines the input and output fuzzy sets.
build_rules()	Combines the four presentation rules with generated rules.
run_case_study()	Evaluates students A–D and compares predicted labels with expected labels.
explain() / make_plots()	Displays fired rules and creates visualisations.

The program can run the illustrative case study by executing `python student_performance_fis.py`. A single prediction can be requested with `python student_performance_fis.py --predict 85 72 6`; adding `--explain` displays membership degrees and the strongest fired rules. The `--plots` option saves the membership functions and control surfaces in an output folder.

8. Results and Discussion

The supplied Python program was executed with its built-in illustrative case study. All four predicted categories matched the expected categories in the presentation.

Student	Attendance	Marks	Study hours/day	Input terms	Score / 100	Predicted
A	92%	88	7.0	High / Good / High	89.6	Excellent
B	52%	52	4.0	Medium / Average /	43.0	Average

				Medium		
C	25%	22	1.0	Low / Poor / Low	13.6	Poor
D	90%	55	7.0	High / Average / High	67.1	Good

Student A receives the highest score because all three indicators fall in strong categories. Student C receives the lowest score because attendance, marks and study hours are low. Student B falls in the average category, while student D demonstrates that high attendance and study time do not automatically imply excellent performance when marks are only average. The results illustrate how several inputs are combined rather than relying on one factor.

The score is the centroid of the aggregated output membership function. The final category is selected by evaluating the output membership functions at that score and choosing the category with the greatest membership.

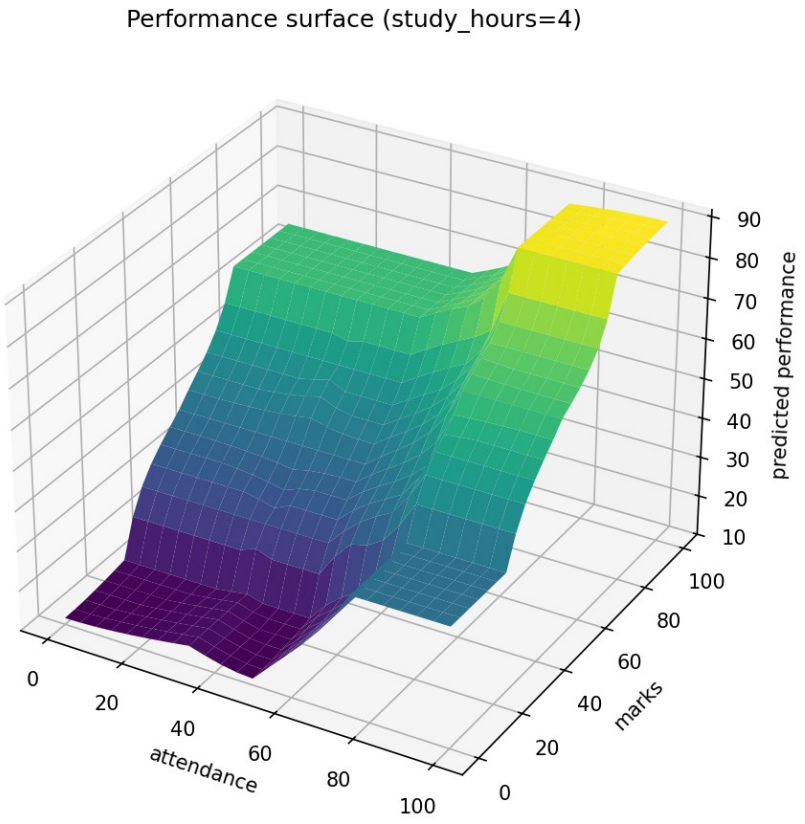


Figure 2. Predicted performance surface against attendance and marks, with study hours fixed at 4.

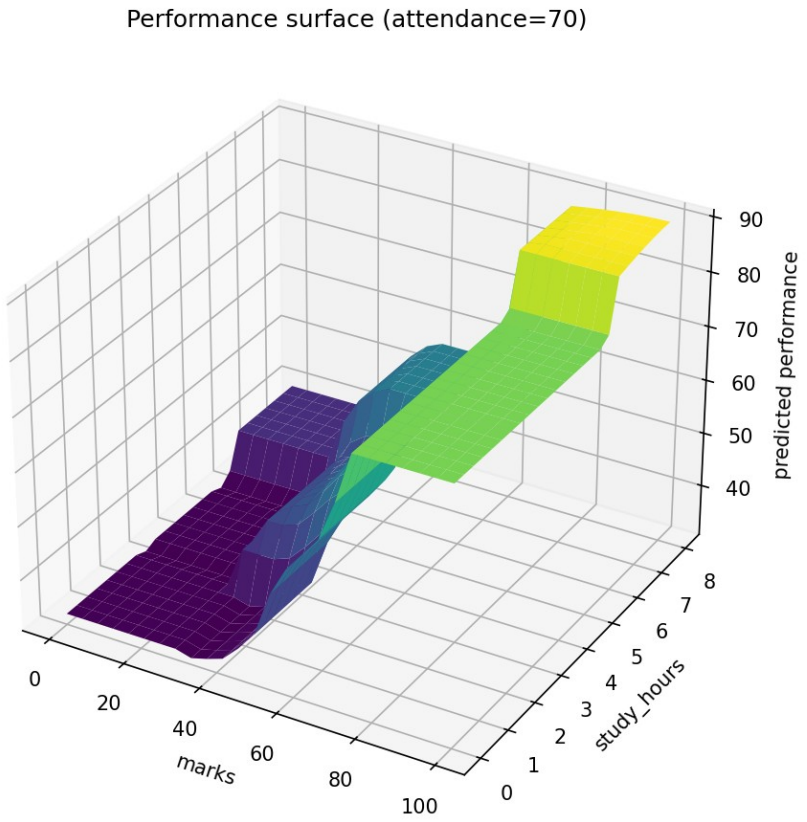


Figure 3. Predicted performance surface against marks and study hours, with attendance fixed at 70%.

9. Advantages and Limitations

Advantages

- Handles gradual transitions through overlapping membership functions.
- Uses readable IF–THEN rules, making the reasoning easier to inspect.
- Combines multiple academic indicators into one output.
- Provides both a numerical score and a linguistic category.
- Can display membership values and fired rules for an individual prediction.

Limitations

- The four-student case study is illustrative and is not actual experimental data.
- Membership-function breakpoints are defaults selected in code, not empirically calibrated values.
- The importance weights and output cut-offs are assumptions; their effect has not been validated against a real dataset.
- The model currently uses only attendance, marks and study hours; assignments and previous-semester performance are not implemented as inputs.
- No accuracy, precision, recall, error rate or comparison against a baseline model is reported.

10. Future Scope

- Calibrate membership functions and rule weights using a real student dataset and domain-expert feedback.
- Include assignment performance, participation and previous-semester academic records.
- Evaluate the model using suitable performance metrics and compare it with conventional machine-learning approaches.
- Build a web or mobile interface for entering student indicators and viewing predictions.
- Conduct validation across different courses, semesters and student groups before practical deployment.

11. Conclusion

The case study demonstrates a working Mamdani Fuzzy Inference System for student performance prediction. It transforms attendance, marks and study hours into fuzzy categories, applies an interpretable rule base and produces a crisp score and performance label through centroid defuzzification. The program successfully reproduces all four expected categories in the presentation's illustrative example. The implementation is therefore a useful demonstration of explainable fuzzy reasoning; however, its predictive effectiveness in real educational settings remains to be established through real data, calibration and validation.

References

1. Senel, A. A., Goksu, B., Senel, E., & Solmaz, F. I. (2025). “Predicting academic performance with fuzzy logic in prospective physical education and sports teachers.” Scientific Reports, 15, 28241. https://doi.org/10.1038/s41598-025-99124-3. (Research methodology cited in the supplied Python source.)

2. Student Performance Prediction Using Fuzzy Logic. Supplied engineering case-study presentation, six slides.

3. student_performance_fis.py. Supplied Python implementation.

Note: The presentation's final slide gives the reference title “Development of Fuzzy Logic Based Student Performance Prediction System,” while the Python source names the Scientific Reports article listed above. The report preserves the more specific bibliographic details provided in the code and flags the difference rather than assuming the titles refer to the same publication.